

Level 1 Challenge 91

▼ Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=0;i<n;i++)
```

Test Case 2

KEYWORD

```
if(arr[i]>max)
```

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

9

Test Case 2

TOKEN COUNT

330

Test Case 3

NLOC

55

```
Hashing1.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXN 200005
5
6 typedef struct {
7     int val;
8     int idx;
9 } Element;
10
11 int cmp(const void *a, const void *b) {
12     Element *e1 = (Element*)a;
13     Element *e2 = (Element*)b;
14     if (e1->val != e2->val)
15         return e1->val - e2->val;
16     return e1->idx - e2->idx;
17 }
18
19 int main() {
20     int n;
21     if (scanf("%d", &n) != 1) return 0;
22
23     int *arr = (int*)malloc(n * sizeof(int));
24     int max = 0;
25     int i;
```

```
26     for(i=0;i<n;i++) {
27         scanf("%d", &arr[i]);
28         if(arr[i]>max) {
29             max = arr[i];
30         }
31     }
32
33     Element *elems = (Element*)malloc(n * sizeof(Element));
34     for (i = 0; i < n; i++) {
35         elems[i].val = arr[i];
36         elems[i].idx = i;
37     }
38     qsort(elems, n, sizeof(Element), cmp);
39
40     int *compressed = (int*)malloc(n * sizeof(int));
41     int distinct_cnt = 0;
```

```
42     for (i = 0; i < n; i++) {
43         if (i == 0 || elems[i].val != elems[i - 1].val) {
44             distinct_cnt++;
45         }
46     }
```

main | CH.SC.U4CSE25225 | Hashing1.c

Ln 89, Col 1 Spaces: 4 UTF-8 CRLF C

```
46     compressed[elems[i].idx] = distinct_cnt - 1;
47 }
48
49 int *first_pos = (int*)malloc(distinct_cnt * sizeof(int));
50 for (i = 0; i < distinct_cnt; i++) first_pos[i] = -1;
51 for (i = 0; i < n; i++) {
52     int c = compressed[i];
53     if (first_pos[c] == -1) {
54         first_pos[c] = i;
55     }
56 }
57
```

```

58  int *seen = (int*)calloc(distinct_cnt, sizeof(int));
59  int *suffix_distinct = (int*)malloc((n + 1) * sizeof(int));
60  suffix_distinct[n] = 0;
61  for (i = n - 1; i >= 0; i--) {
62      int c = compressed[i];
63      if (!seen[c]) {
64          seen[c] = 1;
65          suffix_distinct[i] = suffix_distinct[i + 1] + 1;
66      } else {
67          suffix_distinct[i] = suffix_distinct[i + 1];
68      }
69  }
70
71  long long total_unique_pairs = 0;
72  for (int u = 0; u < distinct_cnt; u++) {
73      int pos = first_pos[u];

```

```
74     if (pos + 1 < n) {
75         total_unique_pairs += suffix_distinct[pos + 1];
76     }
77 }
78
79 printf("%lld\n", total_unique_pairs);
80
81 free(arr);
82 free(elems);
83 free(compressed);
84 free(first_pos);
85 free(seen);
86 free(suffix_distinct);
87 return 0;
88}
89
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
5
1 2 2 1 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing1.exe
6
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
7
1 4 1 2 2 1 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing1.exe
10
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

Problem Description:

Canthi and Sami are having a game! The game is extremely similar to chess, but there is only one piece on the board, which is the Queen. In addition, Queen may only go to the top left corner.

For clarification, If Queen is placed at i,j then in a turn queen can move:

- 1) Any number of cells leftwards.
- 2) Any number of cells upwards.
- 3) Any number of cells Diagonally(only N-W direction).

Please note that board is quarter infinite i.e there is a top left corner but it extends to infinity in south and east direction..

Functional Description

- 1) Canthi will always play the first turn.
- 2) They both will get alternative turns.
- 3) They must move the queen in their turn (No skip option) and that too according to the rule specified above.
- 4) Whosoever is unable to make a move loses.

Given The position of queen on the board(0 index based). Print who will win the game.

Constraints:

$$1 \leq i \leq 10000$$

$$0 \leq a, b \leq 1000000$$

Input:

First line of input contains an integer t - no of test cases.

Each test case is described as a line containing two integers a and b . which is the position of queen on the board.

Output:

print the name of person who will win the game.

Logical Test Cases

Test Case 1

INPUT (STDIN)

2
1 2
2 3

EXPECTED OUTPUT

sami
canthi

Test Case 2

INPUT (STDIN)

3
5 6
1 5
2 3

EXPECTED OUTPUT

canthi
canthi
canthi

Mandatory Test Cases

Test Case 1

KEYWORD

while(t--)

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

6

Test Case 2

TOKEN COUNT

210

Test Case 3

NLOC

36

Hashing2.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4
5 int main() {
6     int t;
7     if (scanf("%d", &t) != 1) return 0;
8
9     double phi = (1.0 + sqrt(5.0)) / 2.0;
10
11     while (t--) {
12         long long a, b;
13         if (scanf("%lld %lld", &a, &b) != 2) break;
14
15         if (a > b) {
16             long long tmp = a;
17
18             b = tmp;
19         }
20
21         long long k = b - a;
22         long long expected_a = (long long)(k * phi);
23
24         if (a == expected_a) {
25             printf("sami\n");
```

main | CH.SC.U4CSE25225 | Hashing2.c

Ln 32, Col 1 Spaces: 4 UTF-8 CRLF C

```
26     } else {  
27         printf("canthi\n");  
28     }  
29 }
```

```
30     return 0;  
31}  
32
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
2
1 2
2 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing2.exe
sami
canthi
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
3
5 6
1 5
2 3
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing2.exe
canthi
canthi
canthi
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

Question description

Alice likes festivals a lot as we all do. He also likes spending money on these festivals. He spends money in buying various things on these festivals. But he has problem of forgetting. He only remembers his top three maximum spendings for any festival.

For eg, on Holi he spends 25 units on colors, 50 units on water sprays, 100 units on gifts, 150 units on sweets but he remebers only his top 3 spendings ,i.e., 50, 100 & 150.

Now as the year ends he wants to know the **festival on which he spent most of his money that he can remember**.

Constraints

$$1 \leq T \leq 100$$

$$1 \leq N \leq 10^4$$

$$1 \leq X \leq 10^9$$

$$1 \leq |S| \leq 10; |S| \text{ denotes string length}$$

S will consist of **lowercase and/or uppercase English aplhabets**

Input Format

First line contains an integer **T**, denoting number of test cases.

Second line contains an integer **N**, denoting number of spendings in a year.

Each of the next **N** lines contain a string **S** and an integer **X** denoting festival name and spending on one of the events of that festival.

Output Format

For each test case, print a single line containing the festival name and its total spending that he remembers.

If there are more than 1 festival with the maximum spending, print the one which has lexicographically smallest name.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

2
6
B 20
A 2
A 10
A 10
B 30
A 30
3
abc 10
xyz 15
oop 8

EXPECTED OUTPUT

A 50
xyz 15

Test Case 2

INPUT (STDIN)

2
5
BA 25
AD 24
Ag 10
AE 15
AT 30
3
abc 15
xyz 25
oops 8

EXPECTED OUTPUT

AT 30
xyz 25

▼ Mandatory Test Cases

Test Case 1

KEYWORD

while(t--)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

7

Test Case 2

TOKEN COUNT

260

Test Case 3

NLOC

46

Hashing3.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAX_FEST 10005
6 #define NAME_LEN 15
7
8 typedef struct {
9     char name[NAME_LEN];
10    long long top3[3];
11    int count;
12} Festival;
13
14int cmp_desc(const void *a, const void *b) {
15    long long v1 = *(const long long *)a;
16    long long v2 = *(const long long *)b;
17    if (v2 > v1) return 1;
18    if (v2 < v1) return -1;
19    return 0;
20}
21
22int find_or_add(Festival *festivals, int *num_festivals, const char *name) {
23    for (int i = 0; i < *num_festivals; i++) {
24        if (strcmp(festivals[i].name, name) == 0) {
25            return i;
```

```

26     }
27 }
28 strcpy(festivals[*num_festivals].name, name);
29 festivals[*num_festivals].count = 0;
30 festivals[*num_festivals].top3[0] = 0;
31 festivals[*num_festivals].top3[1] = 0;
32 festivals[*num_festivals].top3[2] = 0;
33 (*num_festivals)++;
34 return (*num_festivals) - 1;
35}
36
37void add_spending(Festival *f, long long amount) {
38     if (f->count < 3) {
39         f->top3[f->count++] = amount;
40         qsort(f->top3, f->count, sizeof(long long), cmp_desc);
41     } else {
42         if (amount > f->top3[2]) {

```

```
43         f->top3[2] = amount;
44         qsort(f->top3, 3, sizeof(long long), cmp_desc);
45     }
46 }
```

main | CH.SC.U4CSE25225 | Hashing3.c

Ln 92, Col 1 Spaces: 4 UTF-8 CRLF C

```
48
49int main() {
50    int t;
51    if (scanf("%d", &t) != 1) return 0;
52
53    Festival *festivals = (Festival *)malloc(MAX_FEST * sizeof(Festival));
54
55    while(t--) {
56        int n;
57        if (scanf("%d", &n) != 1) break;
58
59        int num_festivals = 0;
```

```

60     for (int i = 0; i < n; i++) {
61         char name[NAME_LEN];
62         long long amount;
63         scanf("%s %lld", name, &amount);
64         int idx = find_or_add(festivals, &num_festivals, name);
65         add_spending(&festivals[idx], amount);
66     }
67
68     char best_name[NAME_LEN] = "";
69     long long best_sum = -1;
70
71     for (int i = 0; i < num_festivals; i++) {
72         long long sum = 0;
73         for (int j = 0; j < festivals[i].count; j++) {
74             sum += festivals[i].top3[j];
75         }
76         if (sum > best_sum) {

```

```
77         best_sum = sum;
78         strcpy(best_name, festivals[i].name);
79     } else if (sum == best_sum) {
80         if (strcmp(festivals[i].name, best_name) < 0) {
81             strcpy(best_name, festivals[i].name);
82         }
83     }
84 }
85
86 printf("%s %lld\n", best_name, best_sum);
87 }
88
89 free(festivals);
90 return 0;
91}
92
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
2
```

```
6
```

```
B 20
```

```
A 2
```

```
A 10
```

```
A 10
```

```
B 30
```

```
A 30
```

```
3
```

```
abc 10
```

```
xyz 15
```

```
oop 8
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing3.exe
```

```
A 50
```

```
xyz 15
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
2
```

```
5
```

```
BA 25
```

```
AD 24
```

```
Ag 10
```

```
AE 15
```

```
AT 30
```

```
3
```

```
abc 15
```

```
xyz 25
```

```
oops 8
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing3.exe
```

```
AT 30
```

xyz 25

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Hashing	Question Information	Level 1	Challenge 94
----	---------	---------	----------------------	---------	--------------

Question description

The students of college XYZ are getting jealous of the students of college ABC . ABC managed to beat XYZ in all the sports and games events. The main strength of the students of ABC is their unity. The students of XYZ decide to destroy this unity. The geeks of XYZ prepared a special kind of perfume. Anyone who inhales this perfume becomes extremely violent. The students of XYZ somehow manage to spread this perfume throughout ABC 's campus atmosphere.

There are N boys $(1, 2, 3, \dots, N)$ and N girls $(1, 2, 3, \dots, N)$ in ABC college. Each boy has a crush on a single girl and each girl has a crush on a single boy. Since the perfume has been inhaled by each and every student of ABC college, every student decides to beat up his/her crush's crush, ie. , if boy x has a crush on girl y and girl y has a crush on boy z , x will beat z up, provided, of course, if x and z is not the same person.

The doctor of ABC college foresees this situation. He cannot stop so many people from beating each other up, however, he can be prepared for the worst-case patient(s). The worst-case patient(s) will be the patient(s) who get(s) beaten up by the maximum number of students. The doctor comes to you for help. He has 2 questions for you :

1. What is the number of beatings received by the worst-case patient(s) ?
2. What is the total number of pairs of students who ended up beating up each other ?

Constraints :

$$1 \leq T \leq 10$$

$$1 \leq N \leq 10^5$$

Input :

The first line comprises of T , the number of test cases. Each test case comprises of 3 lines. The first line consists of N .

The next line consists of N space separated natural numbers between 1 and N inclusive such that the i th number denotes the the crush of boy i .

The next line consists of N space separated natural numbers between 1 and N inclusive such that the i th number denotes the the crush of girl i .

Output :

For every test case, on a new line, print two space-separated integers, the answer to doctor's question 1 followed by answer to doctor's question 2.

Logical Test Cases

Test Case 1

INPUT (STDIN)

2
3
2 2 1
3 2 1
4
2 3 4 1
2 3 4 1

EXPECTED OUTPUT

1 0
1 4

Test Case 2

INPUT (STDIN)

2
4
4 2 1 3
4 2 1 2
3
2 4 1
2 4 1

EXPECTED OUTPUT

1 0
1 1

Mandatory Test Cases

Test Case 1

KEYWORD

while(true)

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

4

Test Case 2

TOKEN COUNT

535

Test Case 3

NLOC

95

Hashing4.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4
5 void solve() {
6     int n;
7     if (scanf("%d", &n) != 1) return;
8
9     int *boy_crush = (int *)malloc((n + 1) * sizeof(int));
10    int *girl_crush = (int *)malloc((n + 1) * sizeof(int));
11    int *boy_target = (int *)calloc(n + 1, sizeof(int));
12    int *girl_target = (int *)calloc(n + 1, sizeof(int));
13    int *boy_in = (int *)calloc(n + 1, sizeof(int));
14    int *girl_in = (int *)calloc(n + 1, sizeof(int));
15
16    for (int i = 1; i <= n; i++) {
17        scanf("%d", &boy_crush[i]);
18    }
19    for (int i = 1; i <= n; i++) {
20        scanf("%d", &girl_crush[i]);
21    }
22
23    for (int x = 1; x <= n; x++) {
24        int g = boy_crush[x];
25        int z = girl_crush[g];
```

```
26     if (x != z) {
27         boy_target[x] = z;
28         boy_in[z]++;
29     }
30 }
31
32 for (int y = 1; y <= n; y++) {
33     int b = girl_crush[y];
34     int w = boy_crush[b];
35     if (y != w) {
36         girl_target[y] = w;
37         girl_in[w]++;
38     }
39 }
40
```

```
41  int max_beatings = 0;
```

main | CH.SC.U4CSE25225 | Hashing4.c

Ln 82, Col 1 Spaces: 4 UTF-8 CRLF C

```
43      if (boy_in[i] > max_beatings) max_beatings = boy_in[i];
44      if (girl_in[i] > max_beatings) max_beatings = girl_in[i];
45  }
46
47  int mutual_pairs = 0;
48  for (int x = 1; x <= n; x++) {
49      int z = boy_target[x];
50      if (z > x && boy_target[z] == x) {
51          mutual_pairs++;
52      }
53  }
54  for (int y = 1; y <= n; y++) {
```

```
55     int w = girl_target[y];
56     if (w > y && girl_target[w] == y) {
57         mutual_pairs++;
58     }
59 }
60
61 printf("%d %d\n", max_beatings, mutual_pairs);
62
63 free(boy_crush);
64 free(girl_crush);
65 free(boy_target);
66 free(girl_target);
67 free(boy_in);
68 free(girl_in);
```

```
69}  
70  
71int main() {  
72    int t;  
73    if (scanf("%d", &t) != 1) return 0;  
74  
75    while(true) {  
76        if (t <= 0) break;  
77        t--;  
78        solve();  
79    }  
80    return 0;  
81}  
82
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

2

3

2 2 1

3 2 1

4

2 3 4 1

2 3 4 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing4.exe

1 0

1 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

2

4

4 2 1 3

4 2 1 2

3

2 4 1

2 4 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing4.exe

1 0

1 0

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Hashing	Question Information	Level 1	Challenge 95
----	---------	---------	----------------------	---------	--------------

Problem Description

You are given a string which comprises of lower case alphabets (a-z), upper case alphabets (A-Z), numbers, (0-9) and special characters like !,.,; etc.

You are supposed to find out **which character occurs the maximum number of times and the number of its occurrence**, in the given string. If two characters occur equal number of times, you have to output the character with the lower **ASCII value**.

For example, if your string was: **aaaaAAAA**, your output would be: **A 4**, because **A has lower ASCII value than a**.

Input format:

The input will contain a string.

Output format:

You've to output two things which will be separated by a space:

- i) The character which occurs the maximum number of times.
- ii) The number of its occurrence.

Constraints:

The maximum length of the string can be **1000**.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) puppy is a dog!!!!!!!!!!!!	INPUT (STDIN) leana is a singer!!!!!!!!
EXPECTED OUTPUT ! 12	EXPECTED OUTPUT ! 7

CH.SC.U4CSE25225 - M ASHWIN | Department of Computer Science & Engineering (CSE) | 2026

▼ Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=0;i<l;i++)
```

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

6

Test Case 2

TOKEN COUNT

210

Test Case 3

NLOC

35

Hashing5.c +

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char s[1005];
6     if (!fgets(s, sizeof(s), stdin)) return 0;
7
8     int l = strlen(s);
9     while (l > 0 && (s[l - 1] == '\n' || s[l - 1] == '\r')) {
10         s[--l] = '\0';
11     }
12
13     int freq[256] = {0};
14     int i;
15     for(i=0;i<l;i++) {
16         freq[(unsigned char)s[i]]++;
```

main | CH.SC.U4CSE25225 | Hashing5.c

Ln 32, Col 1 Spaces: 4 UTF-8 CRLF C

```
18
19     int max_cnt = -1;
20     char best_char = 0;
21
22     for (i = 0; i < 256; i++) {
23         if (freq[i] > max_cnt) {
24             max_cnt = freq[i];
25             best_char = (char)i;
```

```
26     }  
27 }  
28  
29 printf("%c %d\n", best_char, max_cnt);
```

```
30     return 0;  
31}  
32
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
puppy is a dog!!!!!!!!!!!!!!
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing5.exe
! 12
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
leana is a singer!!!!!!
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing5.exe
! 7
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab> 
```

Problem Description

Jenish and Neha are excellent friends. After performing several queries, Neha challenges Jenish to determine the highest possible Rating of the provided array A . According to Neha, the highest occurrence of an element in an array is its rating.

Jenish is given M and Q , the Magical Numbers. Jenish may perform Addition or Subtraction with M at most Q times for each element in the supplied Array A .

Because Jenish is stumped and unable to discover a solution, assist him in determining the highest possible ratings for the given array after applying queries to each element.

Constraints :

- $1 \leq N \leq 1000000$
- $1 \leq M \leq 100$
- $1 \leq Q \leq 10$
- $1 \leq A_i \leq 1000000$

Input :

- First line of Input contains integer M
- Second line contains integer Q
- Third line contains integer N
- Fourth line contains N elements representing elements of array A

Output :

- Output the highest possible rating of array A after applying Queries.

Explanation for test case 1

Jenish can add 1 in 1st element and subtract 1 from 3rd element to get highest frequency of 2 , i.e. 3

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

1
1
4
1 2 3 4

EXPECTED OUTPUT

3

Test Case 2

INPUT (STDIN)

1
1
5
11 12 31 41 51

EXPECTED OUTPUT

2

▼ Mandatory Test Cases

Test Case 1

KEYWORD

for(i=0;i<N;i++)

Test Case 2

KEYWORD

hash[A[i]]++;

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

9

Test Case 2

TOKEN COUNT

300

Test Case 3

NLOC

48

Code Editor

c

1 #include <stdio.h>
2 int main()

Custom Input (stdin)

T1 T2

```
Hashing6.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int cmp_int(const void *a, const void *b) {
5     return (*(const int *)a - *(const int *)b);
6 }
7
8 int main() {
9     int M, Q, N;
10    if (scanf("%d %d %d", &M, &Q, &N) != 3) return 0;
11
12    int *A = (int *)malloc(N * sizeof(int));
13    int *hash = (int *)calloc(1000005, sizeof(int));
14    int i;
15    for(i=0;i<N;i++) {
16        scanf("%d", &A[i]);
17        hash[A[i]]++;
18    }
19
20    int **groups = (int **)malloc(M * sizeof(int *));
21    int *group_sizes = (int *)calloc(M, sizeof(int));
22    int *group_caps = (int *)calloc(M, sizeof(int));
23
24    for (i = 0; i < M; i++) {
25        group_caps[i] = 16;
```

```
26     groups[i] = (int *)malloc(group_caps[i] * sizeof(int));
27 }
28
29 for (i = 0; i < N; i++) {
30     int rem = A[i] % M;
31     int q = A[i] / M;
32     if (group_sizes[rem] == group_caps[rem]) {
33         group_caps[rem] *= 2;
34         groups[rem] = (int *)realloc(groups[rem], group_caps[rem] * sizeof(int));
```

main | CH.SC.U4CSE25225 | Hashing6.c

Ln 68, Col 1 Spaces: 4 UTF-8 CRLF C

```
36     groups[rem][group_sizes[rem]++] = q;
37 }
38
39 int max_rating = 0;
40 for (int r = 0; r < M; r++) {
```

```
41     if (group_sizes[r] == 0) continue;
42     qsort(groups[r], group_sizes[r], sizeof(int), cmp_int);
43
44     int left = 0;
45     for (int right = 0; right < group_sizes[r]; right++) {
46         while (groups[r][right] - groups[r][left] > 2 * Q) {
47             left++;
48         }
49         int current_len = right - left + 1;
50         if (current_len > max_rating) {
51             max_rating = current_len;
52         }
53     }
54 }
```

```
55
56     printf("%d\n", max_rating);
57
58     for (i = 0; i < M; i++) {
59         free(groups[i]);
60     }
61     free(groups);
62     free(group_sizes);
63     free(group_caps);
64     free(hash);
65     free(A);
66     return 0;
67 }
68
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

1

1

4

1 2 3 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing6.exe

3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

1

1

5

11 12 31 41 51

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing6.exe

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Hashing	Question Information	Level 1	Challenge 97
----	---------	---------	----------------------	---------	--------------

Problem Description

Given an array A of N integers. Now, you have to output the sum of unique values of the maximum subarray sum of all the possible subarrays of the given array A .

Note: Subarray means contiguous elements with at-least one element in it.

Constraints

$1 \leq N \leq 2000$

$0 \leq |A_i| \leq 10^9$

Input Format

The first line of the input contains a single integer N , the total number of elements in array A .
The next line of the input contains N space-separated integers representing the elements of the array.

Output Format

The only single line of the output should contain a single integral value representing the answer to the problem.

Logical Test Cases

Test Case 1

INPUT (STDIN)

4
5 -2 7 -3

EXPECTED OUTPUT

17

Test Case 2

INPUT (STDIN)

15
2 3 4 5 6 7 3 4 2 7 6 8 6 4 9

EXPECTED OUTPUT

2021

▼ Mandatory Test Cases

Test Case 1

KEYWORD

```
int NA[N];
```

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

3

Test Case 2

TOKEN COUNT

555

Test Case 3

NLOC

73

Hashing7.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int cmp_ll(const void *a, const void *b) {
5     long long v1 = *(const long long *)a;
6     long long v2 = *(const long long *)b;
7     if (v1 < v2) return -1;
8     if (v1 > v2) return 1;
9     return 0;
10}
11
12int main() {
13     int N;
14     if (scanf("%d", &N) != 1) return 0;
15
16     int NA[N] ;
17     for (int i = 0; i < N; i++) {
18         scanf("%d", &NA[i]);
19     }
20
21     long long total_subs = (long long)N * (N + 1) / 2;
22     long long *max_sums = (long long *)malloc(total_subs * sizeof(long long));
23     long long count = 0;
24
25     for (int l = 0; l < N; l++) {
```

```
26     long long max_so_far = NA[1];
27     long long curr_max = NA[1];
28     max_sums[count++] = max_so_far;
29
```

main | CH.SC.U4CSE25225 | Hashing7.c Ln 57, Col 1 Spaces: 4 UTF-8 CRLF C

```
30     for (int i = 1 + 1; i < N; i++) {
31         if (curr_max + NA[r] > NA[r]) {
32             curr_max = curr_max + NA[r];
33         } else {
34             curr_max = NA[r];
35         }
36         if (curr_max > max_so_far) {
```

```
37         max_so_far = curr_max;
38     }
39     max_sums[count++] = max_so_far;
40 }
41 }
42
43 qsort(max_sums, count, sizeof(long long), cmp_ll);
44
45 long long total_unique_sum = 0;
46 for (long long i = 0; i < count; i++) {
47     if (i == 0 || max_sums[i] != max_sums[i - 1]) {
```

```
48         total_unique_sum += max_sums[i];
49     }
50 }
51
52 printf("%lld\n", total_unique_sum);
53
54 free(max_sums);
55 return 0;
56}
57
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

4

5 -2 7 -3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing7.exe

17

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

15

2 3 4 5 6 7 3 4 2 7 6 8 6 4 9

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing7.exe

2021

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Hashing	Question Information	Level 1	Challenge 98
----	---------	---------	----------------------	---------	--------------

Question description

You are given an array A of length N which is initialised with 0 . You will be given Q queries of two types:

- 1 k : set value 1 at index k in array A
- 2 y : print the smallest index x which is greater than or equal to y and having value 1 . If there is no such index print -1 .

Note: Indexing is 1 based

Constraints

- $1 \leq n \leq 10^9$
- $1 \leq q \leq 5 * 10^5$
- $1 \leq y, k \leq n$

Input Format

First line contains two integers N and Q separated by a space.

The next Q lines contain the type of query (i.e. either a 1 or a 2), then a space, then for type 1 queries integer k and for type 2 queries integer y

Output Format

For each query type 2 , print in new line, the smallest index x which is greater than or equal to y and having value 1 . If there is no such index print -1 .

Explanation for Test case 1

For first query: 2 3, there is no index greater than or equal index 3, having value -1, so the answer is -1.

For second query: 1 2, set value -1 at index 2.

For third query: 2 1, index 2 is greater than index 1 , having value -1, so the answer is 2.

For fourth query: 2 3, there is no index greater than or equal index 3, having value -1, so the answer is -1.

For fifth query: 2 2, index 2 is equal to index 2 , having value -1, so the answer is 2.

▼ Logical Test Cases

Test Case 1
INPUT (STDIN)
5 5 2 3 1 2 2 1 2 3 2 2
EXPECTED OUTPUT
-1 2 -1 2

Test Case 2
INPUT (STDIN)
5 5 2 1 1 2 2 2 1 2 2 3
EXPECTED OUTPUT
-1 2 -1

▼ Mandatory Test Cases

Test Case 1
KEYWORD
for(i=0;i<q;i++)

▼ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY
7

Test Case 2
TOKEN COUNT
225

Test Case 3
NLOC
38

```
Hashing8.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node {
5     int key;
6     int priority;
7     struct Node *left;
8     struct Node *right;
9 } Node;
10
11 Node* create_node(int key) {
12     Node* n = (Node*)malloc(sizeof(Node));
13     n->key = key;
14     n->priority = rand();
15     n->left = NULL;
16     n->right = NULL;
17     return n;
18 }
19
20 void split(Node* t, int key, Node** l, Node** r) {
21     if (!t) {
22         *l = *r = NULL;
23     } else if (key < t->key) {
24         split(t->left, key, l, &t->left);
25         *r = t;
```

```

26     } else {
27         split(t->right, key, &t->right, r);
28         *l = t;
29     }
30}
31
32void merge(Node** t, Node* l, Node* r) {
33    if (!l || !r) {
34        *t = l ? l : r;
35    } else if (l->priority > r->priority) {
36        merge(&l->right, l->right, r);
37        *t = l;
38    } else {
39        merge(&r->left, l, r->left);
40        *t = r;
41    }
42}
43
44int find_val(Node* t, int key) {
45    while (t) {

```

```

46     if (t->key == key) return 1;
47     if (key < t->key) t = t->left;
48     else t = t->right;
49 }
50 return 0;
51}
52

```

main | CH.SC.U4CSE25225 | Hashing8.c

Ln 103, Col 1 Spaces: 4 UTF-8 CRLF C

```

53void insert_key(Node* root, int key) {
54     if (find_val(*root, key)) return;
55     Node *l, *r;
56     split(*root, key, &l, &r);
57     Node* n = create_node(key);
58     merge(&l, l, n);
59     merge(root, l, r);
60}
61
62int lower_bound(Node* root, int key) {
63     int ans = -1;
64     Node* curr = root;
65     while (curr) {

```

```
66     if (curr->key >= key) {
67         ans = curr->key;
68         curr = curr->left;
69     } else {
70         curr = curr->right;
71     }
72 }
73 return ans;
74}
75
76void free_tree(Node* t) {
77     if (!t) return;
78     free_tree(t->left);
79     free_tree(t->right);
80     free(t);
81}
82
83int main() {
84     int n, q;
```

```

85     if (scanf("%d %d", &n, &q) != 2) return 0;
86
87     Node* root = NULL;
88     int i;
89     for(i=0;i<q;i++) {
90         int type, val;
91         scanf("%d %d", &type, &val);
92         if (type == 1) {
93             insert_key(&root, val);
94         } else if (type == 2) {
95             int ans = lower_bound(root, val);
96             printf("%d\n", ans);
97         }
98     }
99
100     free_tree(root);
101     return 0;
102 }
103

```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5 5

2 3

1 2

2 1

2 3

2 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing8.exe

-1

2

-1

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5 5

2 1

1 2

2 2

1 2

2 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing8.exe

-1

2

-1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Hashing	Question Information	Level 1	Challenge 99
----	---------	---------	----------------------	---------	--------------

Problem Description

There are N chocolates denoted by array A where $A[i]$ is the length of the i -th chocolate.

Alice can melt each chocolate and then convert it into a chocolate whose length is any divisor of the number $A[i]$.

So, a chocolate of length $A[i]$ can be converted into X different types of chocolate where X is the count of divisors of the number $A[i]$.

So you need to count the total unordered pair of chocolates such that their X value is same.

Constraints

$1 \leq N \leq 10^5$
 $1 \leq A[i] \leq 10^6$

Input Format

The first line contains an integer N as input denoting the total number of elements in the array A .

The next line contains N space-separated integers that denote the elements of the array A .

Output Format

In the output, print the total number of ways as mentioned in the statement.

▼ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 3 2 3 4 EXPECTED OUTPUT 1	INPUT (STDIN) 10 2 3 4 2 7 6 8 6 4 9 EXPECTED OUTPUT 12
CH.SC.U4CSE25225 - M ASHWIN Department of Computer Science & Engineering (CSE) 2026	

✓ Mandatory Test Cases

Test Case 1

KEYWORD

`while(--N)`

✓ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

8

Test Case 2

TOKEN COUNT

345

Test Case 3

NLOC

44

```
Hashing9.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX_VAL 1000005
5
6 int count_divisors(int n) {
7     int count = 0;
8     for (int i = 1; i * i <= n; i++) {
9         if (n % i == 0) {
10             if (i * i == n) count++;
11             else count += 2;
12         }
13     }
14     return count;
15}
16
17int main() {
18     int N;
19     if (scanf("%d", &N) != 1) return 0;
20
21     int original_N = N;
22     int *A = (int *)malloc(original_N * sizeof(int));
23     for (int i = 0; i < original_N; i++) {
24
25     }
```

```
26
27     int freq_div[1000] = {0};
28     int i = 0;
29     freq_div[count_divisors(A[i++])]++;
30     while(--N) {
31         freq_div[count_divisors(A[i++])]++;
32     }
33
34     long long total_pairs = 0;
35     for (int d = 0; d < 1000; d++) {
36         if (freq_div[d] > 1) {
```

```
37         total_pairs += (long long)freq_div[d] * (freq_div[d] - 1) / 2;
38     }
39 }
40
41 printf("%lld\n", total_pairs);
42
43 free(A);
44 return 0;
45}
46
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
3
2 3 4
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing9.exe
1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
10
2 3 4 2 7 6 8 6 4 9
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing9.exe
12
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

Problem Description

Mathison has uncovered an ancient piece of paper on which are written N integers. This set of numbers will be referred to as A . Mathison learned in History class that a group of integers is unique if and only if their total is divisible by a mythological constant M .

Mathison tries to find out how many **distinct** triplets of numbers, from the piece of paper, have their sum divisible by M . Unfortunately, this problem is quite hard to crack and he needs your help.

Constraints

- $1 \leq N \leq 2 \times 10^5$
- $1 \leq M \leq 10^4$
- $0 \leq A[i] \leq 2 \times 10^9$

Input

The first line of the input file contains two space-separated integers, N and M , representing the number of integers and the mythical constant.

The next line contains N space-separated integers, where the i^{th} integer represents $A[i]$.

Output

The output file should contain only one integer, the answer to Mathison's question.

Explanation for Test case 1

There are 26 *special*/trios: {0, 1, 2}; {0, 1, 8}; {0, 2, 5}; {0, 2, 6}; {0, 2, 9}; {0, 3, 7}; {0, 5, 8}; {0, 6, 8}; {0, 8, 9}; {1, 2, 7}; {1, 3, 4}; {1, 5, 6}; {1, 5, 9}; {1, 6, 9}; {1, 7, 8}; {2, 4, 8}; {2, 5, 7}; {2, 6, 7}; {2, 7, 9}; {3, 4, 5}; {3, 4, 6}; {3, 4, 9}; {5, 6, 9}; {5, 7, 8}; {6, 7, 8}; {7, 8, 9};

Note: Here we only show the **positions** (0-indexed) in each trio.

Logical Test Cases

Test Case 1

INPUT (STDIN)

10 5
1 10 4 3 2 5 0 1 9 5

EXPECTED OUTPUT

26

Test Case 2

INPUT (STDIN)

10 5
11 10 14 31 21 15 10 11 9 51

EXPECTED OUTPUT

31

Mandatory Test Cases

Test Case 1

KEYWORD

while(i<k)

▼ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY
10

Test Case 2
TOKEN COUNT
450

Test Case 3
NLOC
55

Hashing10.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int n, m;
6     if (scanf("%d %d", &n, &m) != 2) return 0;
7
8     long long *cnt = (long long *)calloc(m, sizeof(long long));
9     for (int idx = 0; idx < n; idx++) {
10         long long val;
11         scanf("%lld", &val);
12         cnt[val % m]++;
13     }
14
15     long long total_triplets = 0;
16
17     for (int i = 0; i < m; i++) {
18         for (int j = i; j < m; j++) {
19             int k = (m - (i + j) % m) % m;
20             if (k < j) continue;
21
22             if (i == j && j == k) {
23
24                 } else if (i == j && j < k) {
25                     total_triplets += (cnt[i] * (cnt[i] - 1) / 2) * cnt[k];
```

main | CH.SC.U4CSE25225 | Hashing10.c

Ln 44, Col 1 Spaces: 4 UTF-8 CRLF C

```
26         } else if (i < j && j == k) {
27             total_triplets += cnt[i] * (cnt[j] * (cnt[j] - 1) / 2);
28         } else if (i < j && j < k) {
29             total_triplets += cnt[i] * cnt[j] * cnt[k];
30         }
31     }
32 }
33
34 int i = 0, k = 1;
35 while(i < k) {
```

```
36     i++;
37 }
38
39 printf("%lld\n", total_triplets);
40
41 free(cnt);
42 return 0;
43}
44
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
10 5
```

```
1 10 4 3 2 5 0 1 9 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing10.exe
```

```
26
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
10 5
```

```
11 10 14 31 21 15 10 11 9 51
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing10.exe
```

```
31
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```